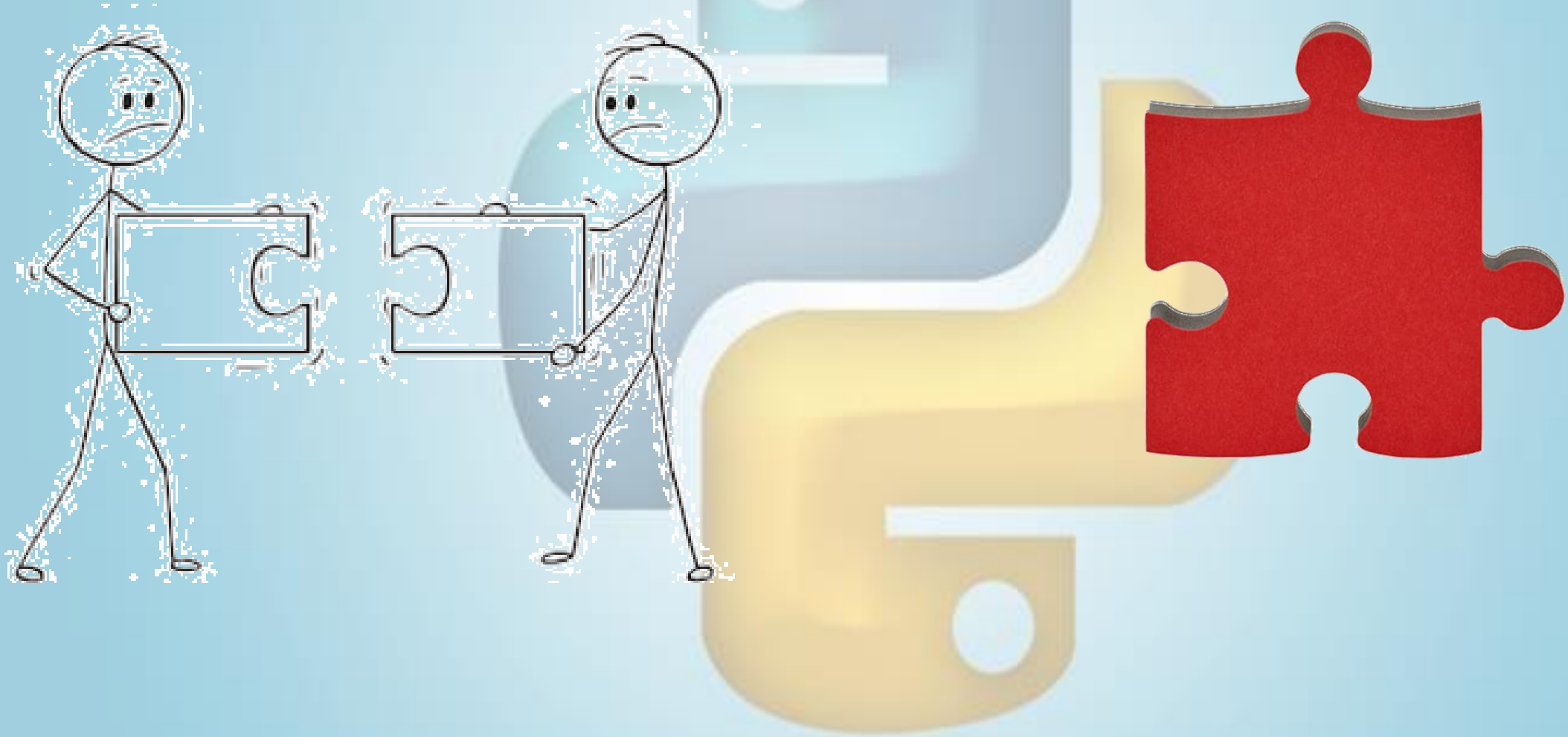


PYTHON PROGRAMMING COURSE

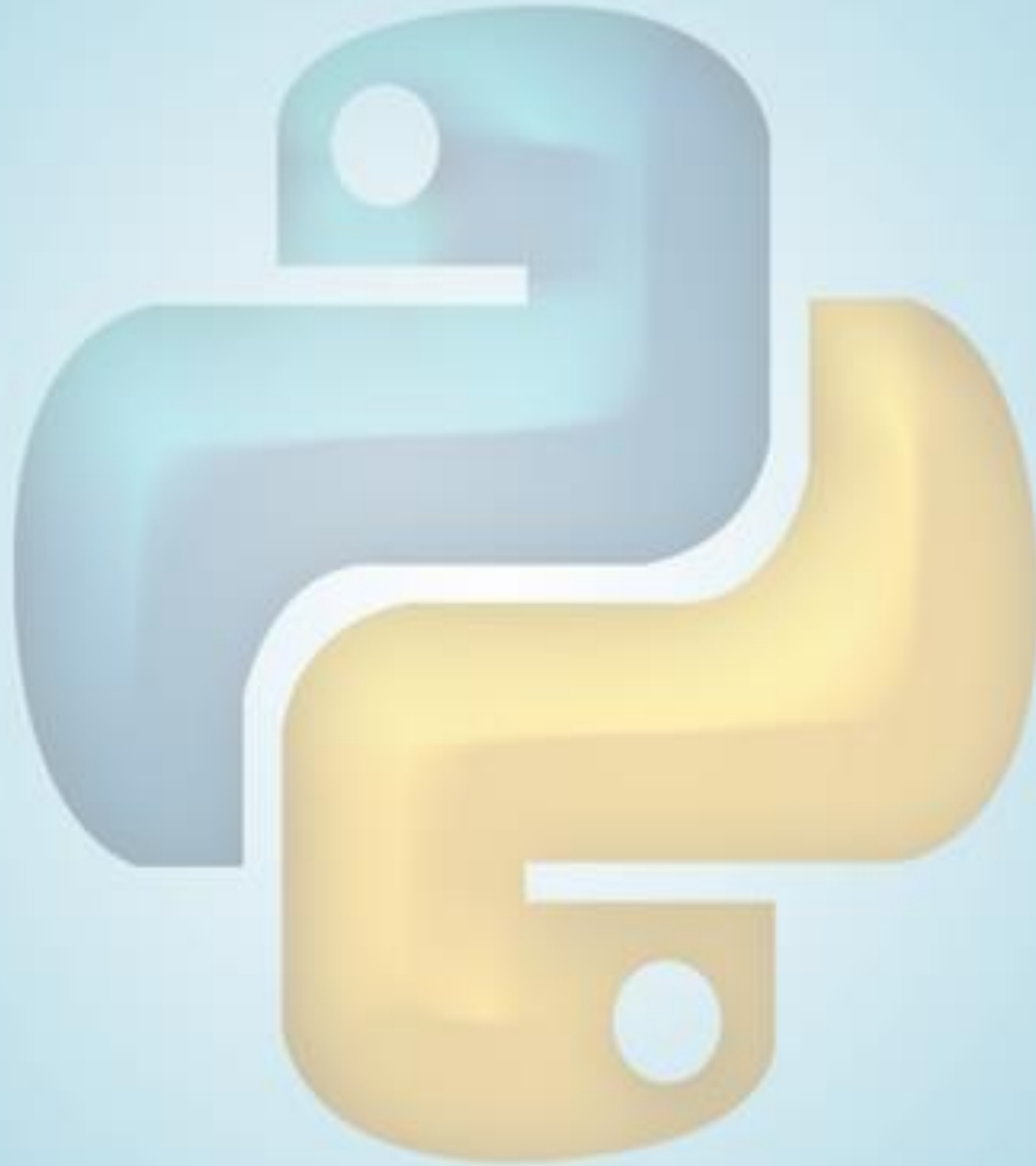


TEAMWORK



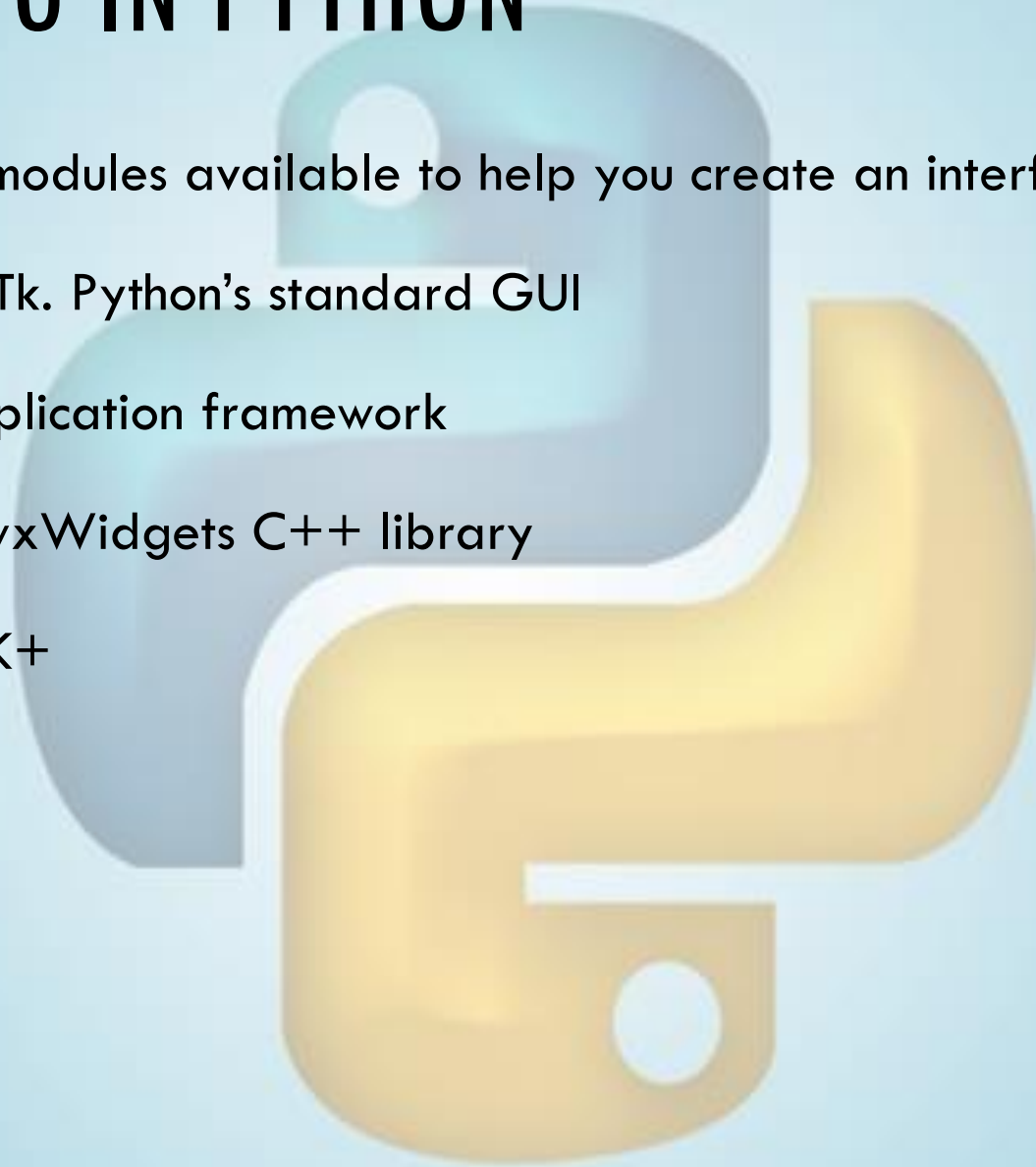
OUTLINE

- GUI programming in Python
- Toolkit
- Tkinter Programming



GUI PROGRAMMING IN PYTHON

- There are a huge number of modules available to help you create an interface
- Tkinter: wrapper around Tcl/Tk. Python's standard GUI
- PyQt: bindings for the Qt application framework
- wxPython: wrapper around wxWidgets C++ library
- pyGTK: wrapper around GTK+
- PyJamas/PyJamas Desktop
- etc.



CHOOSING A TOOLKIT

Toolkit	Important Points
Tkinter	- Limited theme support - Relatively limited widget options. - Bundled with Python since the beginning. - Most commonly used.
PyQt	- Limited licensing choices. - Very good docs - Very beautiful applications.
wxPython	- Very large user base. - Great widget selection. - Bad docs. - Not Python 3 compatible
pyGTK	- Originally developed for GIMP. - Stable with a full selection of widgets. - Only offers theme-able native widgets on Windows + Linux. Mac lags behind somewhat. - Some quirks to work around (some C programming style).

GETTING PYQT5

- 1. Go to <https://www.riverbankcomputing.com/software/sip/download> and grab the latest version of SIP.
- 2. Run the following commands to build and install SIP (version numbers may be different).

```
$ gunzip sip-4.18.tar.gz  
$ tar -xvf sip-4.18.tar  
$ cd sip-4.18/  
$ python configure.py  
$ make  
$ sudo make install
```



USING THE TKINTER MODULE

- Tkinter is Python's default GUI library
- It is based on the Tk toolkit, originally designed for the Tool Command Language (Tcl)
- Due to Tk's popularity, it has been ported to a variety of other scripting languages, including Perl (Perl/Tk), Ruby (Ruby/Tk), and Python (Tkinter)
- Tkinter is not necessarily turned on by default on your system
- You can determine whether Tkinter is available for your Python interpreter by attempting to import the Tkinter module (in Python 1 and 2; renamed to tkinter in Python 3)
- ```
>>> import tkinter
```

# USING THE TKINTER MODULE

## Program 14-1 (empty\_window1.py)

```
1 # This program displays an empty window.
2
3 import Tkinter tkinter
4
5 def main():
6 # Create the main window widget.
7 main_window = Tkinter.Tk()
8
9 # Enter the Tkinter main loop.
10 Tkinter.mainloop()
11
12 # Call the main function.
13 main()
```





# TKINTER PROGRAMMING

■ Five main steps that are required to get tkinter GUI up and running:

1. Import the tkinter module
2. Create a top-level windowing object that contains your entire GUI application.
  - `>>> top = tkinter.Tk() # or just Tk() with "from tkinter import *"`
  - `tkinter.TK()` is root window
3. Build all your GUI components (and functionality) on top (or within) of your top-level windowing object
  - GUI components = text labels, buttons, list boxes, etc. These individual little GUI components are known as widgets
4. Connect these GUI components to the underlying application code
  - Widgets have associated behaviors, such as when a button is pressed, mouse clicked etc
  - These types of user behaviors are called events, and the GUI's response to such events are known as callbacks.
5. Enter the main event loop
  - Widgets are added to the screen by geometry manager
  - Finally, GUI applications enter their server-like infinite loop
  - `>>> top.mainloop() or tkinter.mainloop()` `#tkinter takes over action from here`

# GEOMETRY MANAGERS

- Placer
  - You provide the size of the widgets and locations to place the widgets
  - Then the manager then places them
  - `>>> button1.place(x=-2,y=2,...)`
- Packer
  - it packs widgets into the correct places
  - For every succeeding widget, it looks for any remaining “real estate” into which to pack the next one
  - `>>> button1.pack()`
  - `>>> button2.pack()`
- Grid
  - It is used to specify GUI widget placement, based on grid coordinates
  - `>>> button1.grid(row=0,column=0)`
  - `>>> button1.grid(row=1,column=0)`

# HELLO WORLD

```
import tkinter
```

#step 1 import tkinter module

```
def main():
```

```
 top=tkinter.Tk()
```

parent window

```
 labell=tkinter.Label(top,text="Hello")
```

```
 labell.pack()
```

```
 tkinter.mainloop()
```

#step 2 create top level window

#step 3 make label widget

#arrange it in its parent

#step 4 no event

#step 5 start the event loop

option

```
main()
```

# TKINTER EVENTS AND BINDINGS

|                   |                                           |
|-------------------|-------------------------------------------|
| <Button-1>        | - left mouse button                       |
| <Button-2>        | - middle mouse button (on 3 button mouse) |
| <Button-3>        | - rightmost mouse button                  |
| <B1-Motion>       | - mouse moved with left button depressed  |
| <ButtonRelease-1> | - left button released                    |
| <Double-Button-1> | - double click on button 1                |
| <Enter>           | - mouse pointer entered widget            |
| <Leave>           | - mouse pointer left the widget           |
| <FocusIn>         | - Keyboard focus moved to a widget        |
| <FocusOut>        | - Keyboard focus moved to another widget  |
| <Return>          | - Enter key depressed                     |
| <Key>             | - A key was depressed                     |
| <Shift-Up>        | - Up arrow while holding Shift key        |
| <Configure>       | - widget changed size or location         |

Mouse  
events

Keyboard  
events

# EVENT HANDLING

- Event sources (widgets) can specify their handlers
- Command handlers
  - use the 'command=' keyword followed by the command you want executed

```
#Modified Hello_World

import tkinter #step 1 import tkinter module

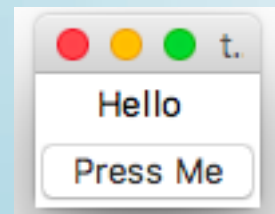
def main():
 top=tkinter.Tk() #step 2 create top level window
 labell=tkinter.Label(top,text="Hello ") #step 3 make label widget
 labell.pack() #arrange it in its parent

 #create another widget
 button1=tkinter.Button(top,text="Press Me",command=top.quit)
 button1.pack(side="left")
 tkinter.mainloop() #step 5 start the event loop

main()
```

command handler

- top.quit/ top.destroy



# CALLBACKS

- callbacks

- A callback is the name of the function that is to be run in response of an event
- • Callbacks can be defined as a free standing function in our program or as a class member.

```
#Modified Hello_World

import tkinter #step 1 import tkinter module

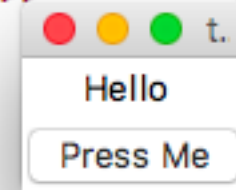
#define callback function here
def quit():
 print("Bye. I am getting out of here")
 import sys
 sys.exit(0)

def main():
 top=tkinter.Tk() #step 2 create top level window
 labell=tkinter.Label(top,text="Hello ") #step 3 make label widget
 labell.pack() #arrange it in its parent

 #create another widget
 button1=tkinter.Button(top,text="Press Me",command=quit)
 button1.pack(side="left")
 tkinter.mainloop() #step 5 start the event loop

main()
|
```

```
>>>
Bye. I am getting out of here
>>>
```



# CALLBACKS WITH CLASS

- Create HelloClass object

```
#Callback with class
#Create HelloClass

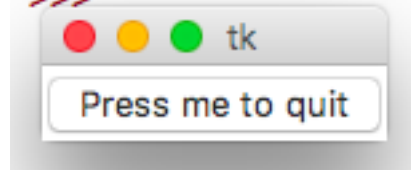
class HelloClass:
 def __init__(self):
 import tkinter
 self.top= tkinter.Tk()
 self.button1 = tkinter.Button(self.top,text="Press me to quit"
 ,command=self.quit)

 self.button1.pack()
 self.top.mainloop()

 def quit(self):
 print("Bye. ")
 import sys
 sys.exit()

HelloClass()
```

```
>>>
Bye.
>>>
```



# BINDING METHODS

#Binding widget with method/event

```
import tkinter
```

```
def hello(event):
 print("Double click to exit")
```

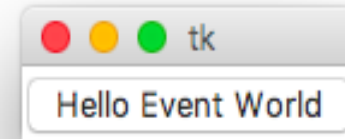
```
def quit(event):
 print("Someone double-clicked! Bye!")
 import sys
 sys.exit()
```

```
top = tkinter.Tk()
button1 = tkinter.Button(top, text="Hello Event World")
button1.pack()
```

#Begin binding

```
button1.bind("<Button-1>",hello)
button1.bind("<Double-1>",quit)
top.mainloop()
|
```

```
Exception in Tkinter callback
Traceback (most recent call last):
 File "/Library/Frameworks/Python.framework/Versions/3.4/lib/python3.4/tkinter/
__init__.py", line 1487, in __call__
 return self.func(*args)
TypeError: hello() takes 0 positional arguments but 1 was given
```



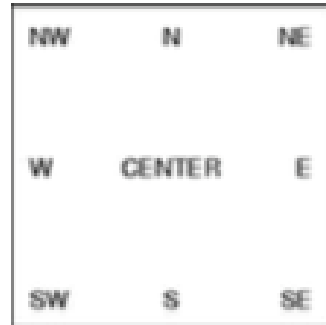


# WIDGETS

| Widget      | Description                                                                                                                          |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Button      | Similar to a Label but provides additional functionality for mouse-overs, presses, and releases, as well as keyboard activity/events |
| Canvas      | Provides ability to draw shapes (lines, ovals, polygons, rectangles); can contain images or bitmaps                                  |
| Checkbutton | Set of boxes, of which any number can be "checked"                                                                                   |
| Entry       | Single-line text field with which to collect keyboard input                                                                          |
| Frame       | Pure container for other widgets                                                                                                     |
| Label       | Used to contain text or images                                                                                                       |
| LabelFrame  | Combo of a label and a frame but with extra label attributes                                                                         |
| Listbox     | Presents the user with a list of choices from which to choose                                                                        |
| Menu        | Actual list of choices "hanging" from a Menubutton from which the user can choose                                                    |
| Menubutton  | Provides infrastructure to contain menus (pulldown, cascading, etc.)                                                                 |
| Message     | Similar to a Label, but displays multiline text                                                                                      |
| PanedWindow | A container widget with which you can control other widgets placed within it                                                         |
| Radiobutton | Set of buttons, of which only one can be "pressed"                                                                                   |
| Scale       | Linear "slider" widget providing an exact value at current setting; with defined starting and ending values                          |
| Scrollbar   | Provides scrolling functionality to supporting widgets, for example, Text, Canvas, Listbox, and Entry                                |
| Spinbox     | Combination of an entry with a button letting you adjust its value                                                                   |
| Text        | Multiline text field with which to collect (or display) text from user                                                               |
| Toplevel    | Similar to a Frame, but provides a separate window container                                                                         |

# STANDARD ATTRIBUTES

- Dimensions
- Colors
- Fonts
- Anchors



- used to define where text is positioned relative to a reference point.

- Relief styles

- refers to certain simulated 3-D effects around the outside of the widget.



- Bitmaps

- used to display a bitmap type: "error", "gray75", "gray50", "gray25", "gray12", "hourglass", "info", "questhead", "question", "warning"

- Cursors



# UPDATING STRINGS

- Widget configuration
  - Configuration can be specified at creation time or as the program is running
  - Use config methods

```
#Widget configuration

import tkinter

top = tkinter.Tk()
labell = tkinter.Label (top, text="Hello world")
labell.pack()
labell.config(text="Hello, Tim")
top.mainloop()
```

- Using text variable

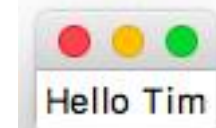
```
#Widget configuration

import tkinter

top = tkinter.Tk()
var = tkinter.StringVar()
labell = tkinter.Label (top, textvariable = var)
labell.pack()

var.set("Hello Tim")
print(var.get())
top.mainloop()
```

Hello Tim



# BUTTON & LABEL WIDGET MODIFIED

```
#Label and Button Widget Modified
```

```
import tkinter
```

```
def quit(top):
 top.destroy()
```

```
top = tkinter.Tk()
```

```
labell = tkinter.Label (top, text= "Hello World")
labell.pack()
```

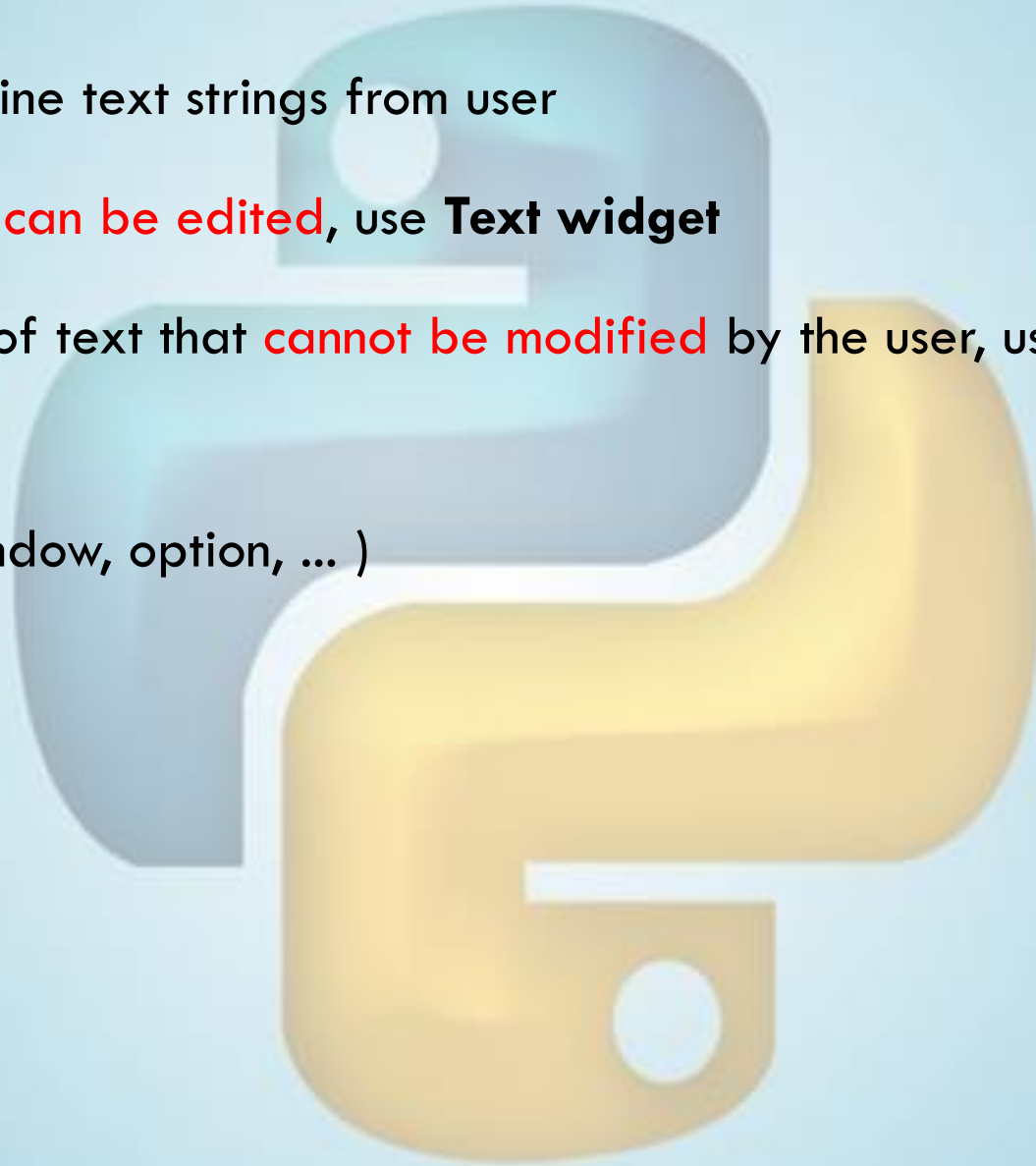
```
button1 = tkinter.Button(top, text="QUIT",command=lambda top=top:quit(top)
 ,bg="red",fg="white")
```

```
#fill => entire space assigned to it, both, X, Y
#assign => additional space to the widget box
```

```
button1.pack(fill =tkinter.X , expand= 1)
top.mainloop()
```

# ENTRY WIDGET

- **Entry widget** accepts single-line text strings from user
- For multiple lines of text that **can be edited**, use **Text widget**
- To display one or more lines of text that **cannot be modified** by the user, use **Label widget**
- Syntax:
  - `w = tkinter.Entry( parent_window, option, ... )`



# GETTING INPUT FROM THE USER

```
#Getting input from user
import tkinter
def quit(top):
 top.destroy()
def display():
 name = var.get()
 label1.configure(text="Hello"+name)

top = tkinter.Tk()
var = tkinter.StringVar("")
entry1 = tkinter.Entry(top, textvariable = var, width=20)
entry1.grid(row=0, column=0)
 #place at row 0, col 0
label1 = tkinter.Label (top, text= "",width=20)
label1.grid(row=0, column=1)

button1 = tkinter.Button(top, text="Display",command = display)
button1.grid(row=1,column=0)
button2 = tkinter.Button(top, text="Quit",command=lambda top=top:quit(top)
 ,bg="red",fg="white")
button2.grid(row=1, column=1)
top.mainloop()
```

# PACK GEOMETRY MANAGER

```
from Tkinter import *
root = Tk()
```

```
w = Label(root, text="Red", bg="red", fg="white").pack(side=LEFT)
w = Label(root, text="Green", bg="green", fg="black").pack(side=LEFT)
w = Label(root, text="Blue", bg="blue", fg="white").pack(side=LEFT)
```

```
mainloop()
```

